AccessAI — Intelligent Accessible Route Planning

Team build guide · 6 members · Execution: Saturday, Oct 17, 2026

1. The Idea

AccessAI is a navigation system that finds the most accessible route for people with disabilities instead of just the shortest one. It avoids stairs and obstacles and prefers ramps and smooth paths.

Three parts work together:

- **Map data** (OpenStreetMap) tells us where footpaths, stairs, ramps and kerbs are.
- **AI image recognition** reads photos and labels ramps, stairs and obstacles.
- **Community feedback** lets users report new barriers so the map stays current.

A routing algorithm combines all three and returns the best path.

2. Scope for Saturday

- One small area (your campus or one neighborhood)
- One user type: wheelchair users
- Route from A to B that avoids stairs and prefers accessible paths
- AI demo on 5-10 sample photos
- A working "Report a barrier" feature

Everything else (more disability types, live data, mobile app) is future work.

3. Tools and Setup (everyone does this first)

Install Python 3.10+ and run:

```
pip install osmnx networkx folium flask scikit-learn transformers torch
pillow
```

Project folder:

```
accessai/
  graph.py      (Member 1)
  routing.py    (Member 2)
  app.py        (Member 3)
  vision.py     (Member 4)
  barriers.py   (Member 5)
  photos/       (Member 5 and 4)
  slides/       (Member 6)
```

Use one shared GitHub repo or a shared folder so everyone can pull each other's files.

4. Work Distribution (6 Members)

- **Member 1, Map Data:** write graph.py, choose the area, download and save the map graph.
- **Member 2, Routing:** write routing.py, the accessibility cost and route finder.
- **Member 3, Web App and Map UI:** write app.py, the page where users enter start and end and see the route.
- **Member 4, AI Image Recognition:** write vision.py, classify photos as ramp, stairs, obstacle or clear path.
- **Member 5, Barrier Reports and Data Collection:** write barriers.py, and collect 10 photos of real stairs, ramps and obstacles in the area.
- **Member 6, Testing, Slides and Demo:** test the routes by walking them, write the limits slide, prepare the demo script and a backup video.

5. How to Create the Map (Member 1)

The map comes from OpenStreetMap. OSM already tags stairs (highway=steps), surfaces and wheelchair access.

Steps

1. Decide the exact place name, for example: "Your College Name, City, Country".
2. Download the walking network for that place.
3. Save it so everyone loads it fast without re-downloading.

graph.py

```
import osmnx as ox
import os

PLACE = 'Your College Name, City, Country'  # CHANGE THIS
FILE = 'map.graphml'

def load_graph():
    if os.path.exists(FILE):
        return ox.load_graphml(FILE)
    G = ox.graph_from_place(PLACE, network_type='walk')
    ox.save_graphml(G, FILE)
    return G

if __name__ == '__main__':
    G = load_graph()
    print('Nodes:', len(G.nodes), 'Edges:', len(G.edges))
```

If the place name is not found, use a center point instead: `ox.graph_from_point((lat, lon), dist=800, network_type='walk')`. Get lat and lon by right-clicking a spot in Google Maps.

Optional: if OSM is missing a ramp or stairs on your campus, add it at openstreetmap.org (Edit) before downloading the data.

6. Accessibility Routing (Member 2)

Idea: give every path segment a "cost". Normal paths cost their length. Stairs and wheelchair=no paths get a huge penalty, so the route avoids them. Rough surfaces get a smaller penalty.

routing.py

```
import osmnx as ox
import networkx as nx

BIG = 10000
ROUGH = ('gravel', 'sand', 'dirt', 'grass', 'cobblestone', 'unpaved')

def edge_cost(d):
    cost = d.get('length', 1)
    hw = d.get('highway')
    if isinstance(hw, list):
```

```
        hw = hw[0]
    if hw == 'steps':
        cost += BIG
    if d.get('wheelchair') == 'no':
        cost += BIG
    if d.get('surface') in ROUGH:
        cost *= 1.5
    return cost

def add_costs(G):
    for u, v, k, d in G.edges(keys=True, data=True):
        d['access_cost'] = edge_cost(d)

def get_route(G, start, end):
    # start and end are (lat, lon) tuples
    a = ox.distance.nearest_nodes(G, X=start[1], Y=start[0])
    b = ox.distance.nearest_nodes(G, X=end[1], Y=end[0])
    route = nx.shortest_path(G, a, b, weight='access_cost')
    return [(G.nodes[n]['y'], G.nodes[n]['x']) for n in route]
```

Test: compare `weight='length'` (normal shortest path) with `weight='access_cost'` and show both on the map. That difference is your best demo moment.

7. Barrier Reports (Member 5)

Users report a blocked path. We save it and add a huge penalty to the nearest path segment, so future routes avoid it.

barriers.py

```
import json
import os
import osmnx as ox

FILE = 'barriers.json'
BIG = 10000

def load_barriers():
    if not os.path.exists(FILE):
        return []
    with open(FILE) as f:
        return json.load(f)
```

```
def add_barrier(lat, lon, note=''):
    data = load_barriers()
    data.append({'lat': lat, 'lon': lon, 'note': note})
    with open(FILE, 'w') as f:
        json.dump(data, f)

def apply_barrier(G, b):
    u, v, k = ox.distance.nearest_edges(G, X=b['lon'], Y=b['lat'])
    G[u][v][k]['access_cost'] += BIG
    if G.has_edge(v, u):
        for kk in G[v][u]:
            G[v][u][kk]['access_cost'] += BIG

def apply_all(G):
    for b in load_barriers():
        apply_barrier(G, b)
```

8. AI Image Recognition (Member 4)

Training a model in 3 days is not realistic. Use a pre-trained model (CLIP) that can classify a photo from labels we give it, with no training.

vision.py

```
from transformers import pipeline

clf = pipeline('zero-shot-image-classification', model='openai/clip-vit-base-
patch32')

LABELS = [
    'a staircase',
    'a wheelchair ramp',
    'a blocked footpath with an obstacle',
    'a smooth clear footpath',
]

def analyze(path):
    results = clf(path, candidate_labels=LABELS)
    best = results[0]
    return best['label'], round(best['score'], 2)
```

```
if __name__ == '__main__':
    import glob
    for p in glob.glob('photos/*.jpg'):
        print(p, analyze(p))
```

The first run downloads the model (needs internet, a few hundred MB). Do this before Saturday, not at the venue. Show the results table in the demo and be honest about mistakes.

9. Web App and Map Display (Member 3)

app.py

```
from flask import Flask, request, redirect, render_template_string
import folium
from graph import load_graph
from routing import add_costs, get_route
from barriers import add_barrier, apply_barrier, apply_all, load_barriers

app = Flask(__name__)
G = load_graph()
add_costs(G)
apply_all(G)

first = list(G.nodes)[0]
CENTER = (G.nodes[first]['y'], G.nodes[first]['x'])

PAGE = '''
<h2>AccessAI - Accessible Route Planner</h2>
<form method="get">
  Start (lat,lon): <input name="start" value="{{start}}" size="28"><br>
  End (lat,lon): <input name="end" value="{{end}}" size="28"><br>
  <button>Find accessible route</button>
</form>
<form method="post" action="/report">
  Barrier (lat,lon): <input name="point" size="28">
  Note: <input name="note">
  <button>Report barrier</button>
</form>
{{ map|safe }}
'''
```

```
def parse(text):
    lat, lon = text.split(',')
    return float(lat), float(lon)

@app.route('/')
def home():
    start = request.args.get('start', '')
    end = request.args.get('end', '')
    m = folium.Map(location=CENTER, zoom_start=17)
    if start and end:
        route = get_route(G, parse(start), parse(end))
        folium.PolyLine(route, color='green', weight=6).add_to(m)
        folium.Marker(route[0], tooltip='Start').add_to(m)
        folium.Marker(route[-1], tooltip='End').add_to(m)
    for b in load_barriers():
        folium.Marker([b['lat'], b['lon']], tooltip=b['note'] or 'Barrier',
                      icon=folium.Icon(color='red')).add_to(m)
    return render_template_string(PAGE, start=start, end=end,
map=m._repr_html_())

@app.route('/report', methods=['POST'])
def report():
    lat, lon = parse(request.form['point'])
    note = request.form.get('note', '')
    add_barrier(lat, lon, note)
    apply_barrier(G, {'lat': lat, 'lon': lon})
    return redirect('/')

if __name__ == '__main__':
    app.run(debug=True)
```

Run it with `python app.py` and open http://127.0.0.1:5000 in the browser. Enter coordinates like `12.9716,77.5946` (use your real campus points).

10. Integration Rules (so the code fits together)

- graph.py exposes `load_graph()`
- routing.py exposes `add_costs(G)` and `get_route(G, start, end)`
- barriers.py exposes `add_barrier`, `apply_barrier`, `apply_all`, `load_barriers`
- vision.py exposes `analyze(path)`
- Coordinates are always `(lat, lon)` order

- Nobody renames a function without telling the group

11. Day-by-Day Plan

- **Wednesday:** everyone installs the tools. Member 1 shares map.graphml with the team. Agree on the area and 2-3 test routes.
- **Thursday:** each member finishes and tests their own file alone. Member 5 collects photos. Member 6 starts slides.
- **Friday:** Member 3 connects all files into the app. Whole team tests: normal route vs accessible route, report a barrier, run the photo demo. Fix bugs. Record a backup demo video.
- **Saturday morning:** one full rehearsal. Keep screenshots and the video ready in case the live demo fails.

12. Demo Script (about 5 minutes)

1. **Problem:** normal maps give the shortest path, which may include stairs or blocked paths.
2. **Idea:** AccessAI plans routes by accessibility.
3. **Demo 1:** show the normal route vs the accessible route on the same trip.
4. **Demo 2:** run the AI on sample photos (ramp, stairs, obstacle).
5. **Demo 3:** report a barrier, then show the route changing to avoid it.
6. **Limits and future work.**

13. Limits and Safety (must be on a slide)

- Map data may be outdated or incomplete.
- AI can misclassify images.
- Photos cannot show ramp slope or path width.
- Routes must be tested with people with disabilities before real use.
- AccessAI must never guarantee that a route is fully safe or accessible. It only suggests more accessible options.

14. Future Work

- Train a custom image model on labeled accessibility photos
- Support more needs: low vision, walker, elderly users
- Add slope, path width and lift/elevator data
- Mobile app with live location and voice guidance
- Moderation and verification for community reports

15. Final Project Summary

AccessAI combines map data, route-planning algorithms, AI image recognition and community feedback into one accessible navigation prototype. The working prototype demonstrates accessible route planning, photo-based barrier detection and real-time user barrier reports for a defined area.